大规模并行有限元组装：从图着色到异构众核寄存器通信的算法演进研究报告

摘要

随着高性能计算（HPC）硬件从多核CPU向众核GPU及异构架构（如申威SW26010）演进，有限元方法（FEM）中的刚度矩阵组装（Assembly）环节已从传统的访存密集型操作转变为极具挑战性的并行同步难题。本报告旨在对并行有限元组装算法进行详尽的梳理与深度剖析，重点关注基于图着色（Graph Coloring）的同步策略、基于原子操作（Atomic Operations）的现代GPU策略、基于域分解（Domain Decomposition）的分布式策略，以及针对国产申威架构的寄存器通信（Register Communication）优化技术。

本研究综合了Cecka, Lew, & Darve (2011)、Markall et al. (2013)、Bangerth et al. (2011)以及Wei Xue等人在Sunway TaihuLight上的最新成果，揭示了算法设计如何随硬件架构的内存层次结构、原子指令性能及通信机制的变化而演变。报告指出，虽然图着色在早期GPU上解决了写冲突问题，但其破坏内存局部性的副作用使其在现代硬件上逐渐被基于Warp聚合的原子操作所取代。而在分布式与异构众核领域，显式的域分解与寄存器级通信成为突破内存墙的关键。

1. 引言：高性能计算下的有限元组装挑战

有限元方法（FEM）作为现代科学工程计算的基石，广泛应用于固体力学、流体力学、电磁学等领域。随着物理模型精度的提升，计算网格的自由度（Degrees of Freedom, DoF）已跨越亿级甚至万亿级门槛。在传统的串行或小规模并行计算中，线性方程组的求解（Solver）通常占据绝大部分计算时间，因此学术界的关注点长期集中在多重网格（Multigrid）、预条件子（Preconditioners）等求解算法的优化上。然而，随着求解器效率的提升以及众核架构对规则计算的加速，矩阵组装——即从单元刚度矩阵构建全局稀疏矩阵的过程——逐渐成为新的性能瓶颈，特别是在非线性、显式动力学或自适应网格细化（AMR）等需要频繁重组装的应用中 。

1.1 组装过程的数学本质与并行困境

从数学角度看，全局刚度矩阵 $\mathbf{K}^G$ 的组装可以表示为单元刚度矩阵 $\mathbf{K}^e$ 的累加：

$$\mathbf{K}^G = \sum_{e=1}^{N_{el}} (\mathbf{L}^e)^T \mathbf{K}^e \mathbf{L}^e$$

其中，$\mathbf{L}^e$ 是将单元局部自由度映射到全局自由度的布尔矩阵。在实际代码实现中，这意味着对于每一个单元 e，程序需要计算其对全局位置 (I, J) 的贡献，并执行累加操作：

$$K_{IJ}^G \leftarrow K_{IJ}^G + K_{ij}^e$$

在并行环境下，多个线程同时处理相邻单元时，极易发生“写冲突”（Write Conflict）或“竞争条件”（Race Condition）。例如，连接多个单元的公共节点（Node），其对应的全局矩阵行会被多个线程同时尝试写入。如果缺乏同步机制，数据将发生覆盖错误。

1.2 硬件架构驱动的算法演进路线

并行组装算法的演进史，本质上是一部硬件架构发展史：

1. **多核CPU时代（Multi-core CPUs）**： 依赖大容量缓存（Cache）和分支预测。算法倾向于直接的“Addto”策略，利用缓存一致性协议处理冲突，尽管这在高并发下会导致缓存抖动 。
2. **早期GPU时代（Tesla/Fermi架构）**： 具有大规模线程并行度，但显存原子操作（Atomic Operations）性能极差。为了避免原子锁带来的串行化，**图着色（Graph Coloring）** 成为主流策略，通过将互不相邻的单元分组执行来从逻辑上消除冲突 。
3. **现代GPU时代（Kepler/Pascal及以后）**： 硬件原子操作单元被下沉至L2甚至L1缓存，性能大幅提升。同时，Warp级原语（Shuffle）的引入使得**Warp聚合（Warp Aggregation）** 成为可能。图着色因其内存访问的不连续性（非合并访问）而被逐渐抛弃，取而代之的是结合局部缓存优化的直接原子组装 。
4. **异构众核时代（申威SW26010）**： 以国产“神威·太湖之光”为代表，采用“管理核

心（MPE）+计算核心阵列（CPEs）”的架构。CPE摒弃了硬件管理的缓存一致性，由用户显式控制高速暂存器（LDM）。在这种架构下，传统的原子操作不可行，算法演进为基于**寄存器通信（Register Communication）** 的显式数据传递与规约 。

---

2. 早期GPU时代的解决方案：图着色策略的兴衰

在2008年至2012年间，GPGPU技术刚刚兴起，硬件对并发写入的支持十分有限。Cecka, Lew, & Darve (2011) 在其发表于 *International Journal for Numerical Methods in Engineering* 的经典论文中，对这一时期的组装算法进行了奠基性的研究 。

2.1 图着色算法原理

图着色策略的核心思想是将数据竞争问题转化为图论中的着色问题。

- **构图：** 将有限元网格抽象为图 $G = (V, E)$，其中顶点 V 代表单元，边 E 代表两个单元共享至少一个节点。
- **着色：** 寻找一个映射 $\mathcal{C} : V \to \{1, \ldots, k\}$，使得对于任意相邻的单元 u, v，有 $\mathcal{C}(u) \neq \mathcal{C}(v)$。
- **执行：** 具有相同颜色 c 的单元集合 S_c 中的所有单元互不共享节点，因此是彼此独立的（Independent Set）。GPU可以启动 k 个连续的内核（Kernel），每个内核只处理一种颜色的单元，从而在无需任何锁或原子操作的情况下保证线程安全。

2.2 Cecka等人的关键贡献与性能分析

Cecka等人实现了基于着色的组装算法，并与串行CPU代码相比取得了 **30倍以上** 的加速比 。然而，他们也敏锐地发现了该方法的致命弱点：

1. 内存访问的碎片化（Memory Fragmentation）：

   在有限元计算中，单元编号通常经过RCM（Reverse Cuthill-McKee）或希尔伯特曲线排序，以保证几何上相邻的单元在内存中也相邻。这极大提升了缓存命中率。

   然而，图着色算法强制将相邻单元分配到不同的颜色组。这意味着在处理“红色”组时，线程束（Warp）内的线程所访问的单元在物理空间上是分散的。这导致了严重的非合并内存访问（Uncoalesced Memory Access）。在GPU上，如果一个Warp

的32个线程访问的地址不连续，内存控制器必须发射多达32次独立的内存事务，导致有效带宽利用率骤降 。

2. 低占有率（Low Occupancy）问题：

   着色算法往往产生不均匀的集合大小。某些颜色可能包含大量单元，而“尾部”颜色可能仅包含少量单元。在处理尾部颜色时，GPU的大量计算核心处于空闲状态，无法掩盖内存延迟。

3. 多项式阶数的影响：

   Cecka的研究还指出，最优策略依赖于单元的多项式阶数 p。对于低阶单元（如线性四面体），计算量少，访存是瓶颈，着色法的访存劣势被放大。对于高阶单元，计算密度增加，访存延迟可以被部分掩盖，着色法的表现相对较好 。

2.3 结论与后续影响

尽管图着色在逻辑上完美解决了竞争问题，但其对硬件缓存机制的破坏使其成为一种“为了正确性牺牲局部性”的妥协方案。随着后续硬件（如NVIDIA Kepler架构）引入了高性能的L2缓存原子操作，图着色的优势迅速丧失。后续研究如Markall et al. (2013) 明确指出，在现代硬件上，着色法的开销往往高于原子操作的冲突开销 。

---

3. 现代GPU架构的标准：原子操作与Warp聚合

2013年以后，随着NVIDIA推出Kepler、Maxwell及后续架构，GPU组装算法发生了范式转移。Markall等人在2013年发表于 *International Journal for Numerical Methods in Fluids* 的论文《Finite element assembly strategies on multi- and many-core architectures》中，详细对比了不同架构下的最优策略 。

3.1 Markall的研究：Addto与Local Matrix Approach (LMA)

Markall等人提出了一对重要的概念对比，揭示了CPU与GPU在组装策略上的根本分歧：

1. **Addto算法（CPU的首选）**：
   - **机制：** 计算一个非零元 K_{ij}^e 后，立即通过间接寻址加到全局矩阵 K_{IJ}^G 中。
   - **适用性：** 适合多核CPU。CPU拥有巨大的L3缓存，可以有效缓冲随机写

入带来的延迟。且CPU线程少，竞争概率低。

- **GPU上的表现**： 极差。随机写入导致显存带宽崩溃，且大量线程竞争导致严重的原子锁冲突。

2. **局部矩阵方法（Local Matrix Approach, LMA）**：
    - **机制**： 每个线程（或线程组）先在本地寄存器或共享内存中完整计算出单元刚度矩阵 $\mathbf{K}^e$，甚至将其临时存储在显存的连续块中。随后，通过一个专门的规约内核（Reduction Kernel）将这些局部矩阵合并到全局矩阵。
    - **优势**： 虽然引入了数据的冗余存储（增加了写操作总量），但所有的写入都是**连续合并（Coalesced）** 的。这完美契合了GPU“吞吐量优先”的特性。
    - **结论**： Markall的研究表明，LMA在GPU上显著优于Addto，且随着GPU计算能力的增强，这种冗余计算+规约的模式比复杂的无冲突算法更具扩展性 。

3.2 Warp聚合（Warp Aggregation）：现代原子操作的极致优化

在最新的GPU组装算法中（如NASA的FUN3D GPU移植工作，以及Sanfui & Sharma 2021的研究），**Warp聚合**被广泛采用以缓解原子操作的压力 。

技术背景： 在CUDA编程模型中，一个Warp包含32个线程，它们以SIMT（单指令多线程）方式执行。在早期的原子操作中，如果Warp内32个线程都要向同一个全局地址（例如由多个单元共享的一个顶点）写入数据，硬件会将其串行化，导致31个周期的等待。

Warp聚合算法流程：

1. **投票与匹配**： Warp内的线程利用 `__shfl_down_sync` 等洗牌指令（Shuffle Intrinsics），在寄存器级别互相交换目标地址 I。
2. **局部规约**： 发现目标地址相同的线程，直接在寄存器中将其值相加。例如，如果线程0, 1, 2都想向地址A写入值，它们会通过洗牌指令将值汇总到线程0。
3. **选举领袖**： 对于每个唯一的目标地址，选出一个“领袖线程”（Leader Thread）。
4. **单一原子操作**： 只有领袖线程向全局内存发起一次 `atomicAdd`。

效果： 这一技术将全局内存的原子事务数量减少了数倍（取决于网格拓扑的连接度），将竞争压力从显存总线转移到了极低延迟的寄存器文件上。对于高阶单元或非结构化网格，这是目前已知最高效的组装策略 。

---

4. 分布式环境下的域分解：deal.II与Ghost Layer管理

当问题规模超出单节点的内存限制时，必须采用基于MPI的分布式计算。此时，组装的挑战从“线程间同步”转变为“进程间通信”。Bangerth, Burstedde, Heister, & Kronbichler (2011) 在 *ACM Transactions on Mathematical Software* 上发表的关于 **deal.II** 库的论文，确立了大规模并行自适应有限元组装的标准范式 。

4.1 “Oracle”抽象与p4est

Bangerth等人引入了一个关键的设计理念：将网格拓扑管理的复杂性剥离出去，交给一个专门的“Oracle”（神谕）处理。在deal.II中，这个Oracle通常由 **p4est** 库担任。

- **p4est的作用：** 它管理着基于八叉树（Octree）的森林结构，负责网格的自适应细化（AMR）、粗化、以及在数万个处理器核心上的负载平衡（基于空间填充曲线，如Z-curve）。
- **解耦：** deal.II不直接维护全局网格连接性，而是向p4est查询：“我拥有哪些单元？我的邻居是谁？”这使得有限元求解器可以专注于代数运算，而无需处理极度复杂的分布式网格重划分逻辑 。

4.2 分布式自由度（DoF）枚举与Ghost Layer

在分布式组装中，核心难题是如何保证跨进程边界的自由度编号一致性。

1. **本地拥有（Locally Owned）与Ghost单元：** 每个MPI进程“拥有”一部分单元，并负责计算这些单元的刚度矩阵。然而，位于子区域边界的单元需要访问邻居进程的数据。这些邻居单元被称为“Ghost Cells”或“Halo Layer”。
2. **Owner-Compute原则：** deal.II 采用“拥有者计算”原则。进程只组装它拥有的单元。对于边界上的共享节点，其对应的全局矩阵行由拥有该节点的进程负责存储。
3. **通信模式：** 在组装过程中，实际上很少进行实时的点对点通信（这太慢了）。相反，deal.II利用Ghost Layer机制：

- 在组装前，通过 `MPI_Isend/Irecv` 交换边界解向量 **u** 的值到Ghost区。
- 组装时，本地计算完全独立，仅读取本地和Ghost数据。
- 组装后，如果使用了非重叠域分解，可能需要一次 `MPI_Allreduce` 或类似的压缩操作来处理界面上的数值累加；但在deal.II的设计中，往往通过精心设计的DoF分配（每个DoF由且仅由一个进程拥有）来最小化这种后处理 。

这种设计使得deal.II能够在 **16,384个处理器核心** 上展现出优异的弱扩展性（Weak Scaling），成功求解了数十亿未知数的复杂多物理场问题 。

---

5. 异构众核架构的特解：申威SW26010与寄存器通信

中国超算“神威·太湖之光”所搭载的 **SW26010** 处理器，代表了另一种极端的架构设计思路。这种架构没有硬件管理的L1/L2缓存一致性，而是提供了用户可控的高速暂存器（LDM）和片上寄存器通信网络。这迫使算法设计必须彻底重构。

5.1 SW26010架构特征

- **核组（Core Group, CG）**： 包含1个管理核心（MPE）和64个计算核心（CPEs）。
- **内存层次**： CPE无法直接高效访问全局内存。每个CPE拥有64KB的 **LDM**（Local Data Memory）。数据必须通过DMA（Direct Memory Access）指令显式搬运。
- **寄存器通信（Register Communication）**： 这是SW26010最独特的特性。CPE阵列呈 8×8 网格排列，硬件支持行与列方向上的低延迟寄存器级数据传输。CPE A可以直接发送数据到CPE B的寄存器，无需经过内存 。

5.2 swParaFEM与Wei Xue团队的突破

针对这一架构，清华大学的Wei Xue团队以及山东省计算中心的Pan等人（swParaFEM）提出了一系列创新算法 。

1. 显式数据流管理：

在swParaFEM中，组装过程被设计为流水线模式。MPE负责任务调度，CPE负责计算。

- **DMA预取**： 在计算单元 i 的同时，DMA引擎异步加载单元 $i+1$ 的坐标和材料数据到LDM。这种“计算-传输重叠”掩盖了内存延迟 。
- 基于寄存器通信的Halo交换（Register-Based Halo Exchange）：

这是对传统组装算法的颠覆。在传统的GPU或CPU上，边界节点的累加依赖于原子操作或共享内存。但在SW26010上，原子操作开销极大。

Xue等人设计了一种基于寄存器通信的规约算法：

- **映射策略**： 将网格映射到 8×8 的CPE阵列上，使得物理上相邻的单元尽量分配给相邻的CPE。
- **通信累加**： 当两个相邻CPE需要对共享边界节点进行累加时，它们不写回主存，而是直接通过寄存器总线传递部分和。
    - 例如，CPE(0,0)计算完局部贡献后，将其发送给CPE(0,1)。CPE(0,1)收到后，加上自己的贡献，再发给下一个。
- **效果**： 这种方法完全绕过了内存系统，实现了接近寄存器速度的通信带宽。Xue团队利用这一技术在隐式求解器中实现了 **1000万核心** 的扩展，达到了近8 PFLOPS的双精度性能，获得了ACM Gordon Bell Prize提名 。

这一案例深刻说明了：在后摩尔定律时代，硬件去除了昂贵的缓存一致性逻辑以换取计算密度，算法设计者必须回归到底层，显式管理数据的流动。

---

6. 无矩阵方法（Matrix-Free）：绕过组装瓶颈的终极方案

面对内存带宽增长远落后于浮点性能增长的现状（Memory Wall），一种更激进的策略正在兴起：**彻底放弃全局矩阵组装**。

6.1 核心理念

无矩阵方法（Matrix-Free）在求解线性方程组 $\mathbf{Ax} = \mathbf{b}$ 时，不需要显式构建矩阵 $\mathbf{A}$。求解器（如共轭梯度法）只需要计算矩阵-向量乘积（MatVec） $\mathbf{v} = \mathbf{Au}$。

根据有限元定义，$\mathbf{Au}$ 可以分解为：

$\mathbf{v} = \sum_e (\mathbf{L}^e)^T (\mathbf{K}^e (\mathbf{L}^e \mathbf{u}))$

即：先从全局向量读取数值到单元（Gather），在单元内部进行积分计算，然后将结果加回全局向量（Scatter）。

6.2 求和因子化（Sum Factorization）

对于高阶单元（$p \geq 4$），传统的单元刚度矩阵极其稠密，组装和存储代价过高。Kronbichler等人（deal.II团队）推广了 求和因子化 技术 。

利用形函数的张量积结构（Tensor Product Structure），三维积分可以分解为三个方向的一维积分序列。这将计算复杂度从 $\mathcal{O}(p^9)$ 降低到 $\mathcal{O}(p^4)$。

- **算术强度提升：** 无矩阵方法虽然增加了计算量（每次MatVec都要重新积分），但它极大地减少了内存访问量（无需读取庞大的稀疏矩阵索引和数值）。这使得算法从“内存受限”转变为“计算受限”，完美契合现代GPU的高FLOPS特性。
- **现状：** 在高阶CFD（如Nek5000, deal.II）应用中，无矩阵方法已成为主流。但在低阶工程应用（$p = 1, 2$）中，由于算术强度不够，传统的组装策略仍然具有优势。

---

7. 总结与展望

7.1 算法与架构的协同演化图谱

7.2 结论与建议

综合文献调研结果，我们可以得出以下结论：

1. **图着色的衰落：** 在现代GPU上，除非为了严格的位级确定性（Bit-wise Reproducibility），否则不应再使用图着色。其带来的内存访问不连续性是性能杀手。
2. **原子操作的胜利：** 结合Warp聚合的原子操作是目前处理低阶非结构化网格组装的

标准方案。它在保持内存合并访问的同时，利用硬件优势解决了竞争问题。

3. **异构架构的启示：** 申威架构的成功证明，通过软件显式管理数据通信（如寄存器通信），可以突破传统缓存一致性协议的扩展性限制。这对于未来Exascale系统的设计具有重要参考价值。
4. **域分解的重要性：** 无论节点内算法如何优化，跨节点的扩展性始终依赖于高效的域分解和Ghost Layer管理。p4est与deal.II的结合展示了代数与几何解耦的优越性。

7.3 未来展望

未来的研究方向将集中在 **混合精度组装（Mixed Precision Assembly）** 上。利用GPU的Tensor Cores进行半精度（FP16/BF16）的单元积分计算，同时在累加阶段保持高精度，有望带来倍数级的性能提升。此外，随着AI for Science的兴起，利用图神经网络（GNN）预测最佳的网格排序或着色方案，也可能为这一古老的问题带来新的生机。

主要参考文献索引：

- Cecka, Lew, & Darve. "Assembly of finite element methods on graphics processors", IJNME, 2011.
- Markall et al. "Finite element assembly strategies on multi- and many-core architectures", IJNMF, 2013.
- Bangerth et al. "Algorithms and Data Structures for Massively Parallel Generic Adaptive Finite Element Codes", ACM TOMS, 2011.
- Pan et al. "swParaFEM: a highly efficient parallel finite element solver on Sunway many-core architecture", J. Supercomput., 2023.
- Yang, Xue et al. "10M-Core Scalable Fully-Implicit Solver...", SC16 (Gordon Bell Prize Winner).
- Kronbichler et al. "Matrix-free finite-element computations...", 2019.